

Final Exam Report

汇报人：李怡伽 | 学号：2023201150 | Kaggle: Yijia Li_1150

模型表现

Model	Type	In Sample	Out of Sample	Cross-validation	Kaggle Score
Data_Refined Mid_term	Price	385399.41	391603.32	400258.98	68.1
	Rent	97671.59	100440.49	99990.33	
ResNet	Price	394826.38	427216.74	\	74.7
	Rent	102749.37	135062.31	\	
Ensemble Tree	Price	242669.83	389954.35	396492.39	76.8
	Rent	79835.13	125059.66	136285.38	
Ensemble Tree+RNN	Price	215525.95	416165.61	\	76.4
	Rent	104292.54	125691.92	\	

数据处理

创新点1: K-Means对位置特征聚类

- 根据板块聚类经纬度，采用肘部法选择n_cluster

创新点2: 语义特征embeddings + K-Means聚类

- 调用nlp模型（text2vec、千问）生成embedding
- 通过K-Means对embedding进行聚类并计算其对每个cluster中心的距离作为特征

废案：Embeddings + PCA降维

```
def nlp_transform(col, df_train, df_val, df_test):
    # 分别将三个集中的文本拿出来，并分别生成nlp向量
    train_text = df_train[col].fillna("").astype(str).tolist()
    train_embed = nlp_model.encode(train_text, batch_size=32, show_progress_bar=True)

    val_text = df_val[col].fillna("").astype(str).tolist()
    val_embed = nlp_model.encode(val_text, batch_size=32, show_progress_bar=True)

    test_text = df_test[col].fillna("").astype(str).tolist()
    test_embed = nlp_model.encode(test_text, batch_size=32, show_progress_bar=True)

    # 此时embeddings过多，因此拟采用K-Means进行聚类
    n_clusters = 5
    kmeans = KMeans(n_clusters = n_clusters, random_state = 111, n_init = "auto")
    kmeans.fit(train_embed) # 在训练集上fit

    # 记录在训练集上每个cluster的中心
    centers = kmeans.cluster_centers_

    # 计算距离特征，默认是使用欧氏距离计算 (L2)
    train_dist = pairwise_distances(train_embed, centers)
    val_dist = pairwise_distances(val_embed, centers)
    test_dist = pairwise_distances(test_embed, centers)
```

残差神经网络模型

创新点1: 学习率调度

- Warm-up 在开始部分线性增加学习率, 防止在初始位置梯度波动过大
- 热身结束后使用CosineAnnealingLR使学习率按余弦曲线下降

创新点2: 加入残差块进行残差链接

- +x操作通过shortcut()完成, 它将残差块的输入直接与输出连接
- 优点: 在深度较大的模型中防止梯度消失, 能够一直更新参数

创新点3: 加入SE注意力模块

- 在残差块后应用, 通过计算出的权重使得模型在噪声维度上“pay less attention”

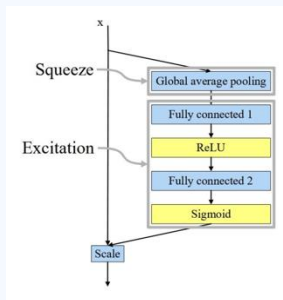
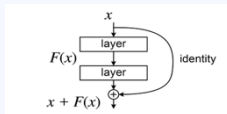
```
# 向前传播: 预测Price
def forward(self, x):
    residual = self.shortcut(x) # 维度不同投影
    out = self.fc1(x)
    out = self.bn1(out)
    out = self.relu(out) # 激活
    out = self.dropout(out)
    out = self.fc2(out)
    out = self.bn2(out)
    out += residual # 残差连接, 防止梯度消失
    out = self.relu(out)

    # 在残差块后应用SE注意力
    if self.use_se:
        out = self.se(out)

    return out
```

```
# SE (Squeeze-and-Excitation) 注意力块: 特征压缩然后还原
class SEBlock(nn.Module):
    def __init__(self, channels, reduction=16): # reduction是压缩倍数
        super(SEBlock, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(channels, max(channels // reduction, 8)), # 把输入压缩到channels整除reduction或者8
            nn.ReLU(inplace=True), # 激活函数
            nn.Linear(max(channels // reduction, 8), channels), # 还原
            nn.Sigmoid() # 压缩到(0,1), 此为注意力权重, 代表每个维度的重要性系数
        )

    def forward(self, x):
        scale = self.fc(x)
        return x * scale
```



集成模型 ver1

创新点: 不同树模型作为Base Model

- Random Forest (效果最差)
- 梯度提升决策树 (GBDT): Xgboost、Catboost、LightGBM (效果最好, 比重最高)

Meta Model: Ridge

- 使用RidgeCV自动选择参数

```
elif model_name == 'lightgbm':
    return {
        'objective': 'regression', # 任务类型为回归
        'metric': 'rmse', # 评估指标
        'learning_rate': 0.015,
        'num_leaves': 96,
        'max_depth': 10,
        'min_child_samples': 20,
        'subsample': 0.8,
        'colsample_bytree': 0.8, # 特征采样
        'reg_alpha': 1.0, # L1正则
        'reg_lambda': 3.0, # L2
        'n_estimators': 2500, # 迭代次数 (树个数上限)
        'random_state': 42,
        'n_jobs': -1,
        'verbosity': -1,
        'force_col_wise': True # 按列构建直方图
```

集成模型ver2

创新点1: 加入初始的神经网络模型捕捉非线性性

- 为避免重复训练, 保存了初始神经网络模型的架构, 此处直接调用

创新点2: 使用LightGBM进行特征选择

- 通过抽样少量样本, 迭代少量次数训练一个轻量的LightGBM模型, 并使用model.feature_importances_取得特征重要性(默认为Gain)
- 最后通过特征重要性排名筛选所用特征

创新点3: 使用Optuna进行超参优化

- 为每个模型设定搜索空间, 采样进行K折交叉验证评估(此处为了提升速度采用2折)
- 从搜索空间中采用TPE方式随机挑选出一组超参

```
if method == 'lightgbm_fast':
    # 使用LightGBM快速计算特征重要性
    # 采样以进一步加速
    if X.shape[0] > 20000:
        print(f" 数据量较大, 使用采样加速(采样20000个样本)...")
        sample_idx = np.random.choice(X.shape[0], 20000, replace=False)
        X_sample = X[sample_idx]
        y_sample = y[sample_idx]
    else:
        X_sample = X
        y_sample = y

    # 使用对数变换的目标值
    y_trans = np.log1p(y_sample)

    # 训练一个快速LightGBM模型获取特征重要性
    model = lgb.LGBMRegressor(
        n_estimators=100, # 只需要少量树就能得到好的特征重要性
        num_leaves=31,
        learning_rate=0.1,
        random_state=42,
        n_jobs=-1,
        verbosity=-1
    )
    model.fit(X_sample, y_trans)

# Optuna调参过程, trial代表本次试验的对象
# 使用lambda是因为objective不止需要传一个参数, 也可以写成函数形式
if self.model_type == 'xgboost':
    objective = lambda trial: self.objective_xgboost(trial, X, y) # objective返回cv_rmse值
elif self.model_type == 'lightgbm':
    objective = lambda trial: self.objective_lightgbm(trial, X, y)
elif self.model_type == 'catboost':
    objective = lambda trial: self.objective_catboost(trial, X, y)
elif self.model_type == 'random_forest':
    objective = lambda trial: self.objective_random_forest(trial, X, y)
elif self.model_type == 'extra_trees':
    objective = lambda trial: self.objective_extra_trees(trial, X, y)
else:
    raise ValueError(f"未知的模型类型: {self.model_type}")

study = optuna.create_study(
    direction='minimize',
    sampler=TPESampler(seed=42) # TPE: 一种采样器, 决定下一次trial试哪个参数
)

study.optimize(objective, n_trials=self.n_trials, show_progress_bar=True)

self.best_params = study.best_params
self.best_score = studv.best_value
```

代码之外

“智星云”平台租用服务器

- Kaggle限额, 人大云调不了Huggingface
- 1.5 元/小时, 排队少, 可选择的种类更多

Vscode上配置链接 使用tmux远程运行代码

- 更加节省时间
- 但只支持.py文件

网课推荐

- 3Blue1Brown@youtube
- 邱锡鹏nlp@b站